Universidad Abierta Interamericana

PROGRAMACIÓN ORIENTADA A OBJETOS

CÓDIGO DE MATERIA 11

Titular: Darío Guillermo Cardacci

Alumna: Morana, Valentina

FACULTAD DE TECNOLOGÍA INFORMÁTICA

UNIVERSIDAD ABIERTA INTERAMERICANA

ÍNDICE

Guía de Revisión Conceptual..... 3

Resolución de Preguntas Conceptuales..... 18

UNIDAD II..... 32

UNIDAD III..... 35

UNIDAD IV..... 39

UNIDAD V..... 40

UNIDAD VI..... 43

GUÍA DE ABORDAJE BIBLIOGRÁFICO..... 46

UNIDAD I..... **46**

Guía de Revisión Conceptual

UNIDAD I

1. Enumere y explique los aspectos más relevantes que hacen que un software de gran magnitud sea complejo.

2. ¿Cuáles son los cinco atributos de un sistema complejo?

3. ¿Cuáles son las dos jerarquías más importantes que consideramos en la orientación a objetos para sistemas complejos?

4. ¿Con qué podemos enfrentar a la complejidad para obtener partes cada vez más pequeñas y simplificadas del dominio del problema?

5. ¿Cuáles son las dos formas de descomposición más conocidas?

6. ¿Explique en qué se diferencia la descomposición algorítmica y la orientada a objetos?

7. ¿Qué rol cumple la abstracción en la orientación a objetos?

8. ¿Qué rol cumple la jerarquía en la orientación a objetos?

9. ¿Consideraría Ud. al diseño orientado a objetos un desarrollo evolutivo o revolucionario? Justifique.

10. ¿Cuántos y cuáles son los modelos básicos que se manejan en el diseño orientado a objetos?

11. ¿Qué es la programación orientada a objetos?

12. ¿Qué es el diseño orientado a objetos?

13. ¿Qué es el análisis orientado a objetos?

14. ¿Cuáles son los elementos fundamentales en el modelo de objetos?

15. ¿Cuáles son los elementos secundarios del modelo de objetos?

16. Explique el significado de la abstracción.

17. Explique el significado del encapsulamiento.

18. Explique el significado de la modularidad.

19. Explique el significado de la jerarquía.

20. Explique el significado de la tipificación.

21. Explique el significado de la concurrencia.

22. Explique el significado de la persistencia.

23. ¿Cómo se denotan las características esenciales de un objeto que lo distinguen de todos los demás tipos de objetos y proporciona así fronteras conceptuales nítidamente definidas respecto a la perspectiva del observador?

24. ¿A qué denominamos un objeto cliente?

25. ¿A qué denominamos un objeto servidor?

26. ¿A qué denomina Meyer el modelo contractual de programación?

27. ¿Qué establece un contrato entre objetos?

28. ¿Cómo se denomina a las formas en que un objeto puede actuar y/o reaccionar, constituyendo estas formas la visión externa completa, estática y dinámica de la abstracción?

29. ¿Cómo se denomina al conjunto completo de operaciones que puede realizar un cliente sobre un objeto, junto con las formas de invocación u órdenes que admite?

30. ¿A qué nos referimos cuando decimos que un concepto central de la idea de abstracción es el de invariancia?

31. ¿Qué se debe definir para cualquier operación asociada a un objeto?

32. ¿Qué es una precondición?

33. ¿Qué es una postcondición?

34. ¿A qué se denomina excepción?

35. ¿A qué se denomina mensaje?

36. ¿El encapsulado es un concepto complementario a la abstracción? Justifique.

37. ¿Cómo se denomina al elemento de un objeto que captura su vista externa?

38. ¿Cómo se denomina al elemento de un objeto que captura su vista interna la cual incluye los mecanismos que consiguen el comportamiento deseado?

39. ¿El concepto de “ocultar los detalles de implementación” lo asociaría a “esconder los detalles de implementación” o a “evitar el uso inapropiado de los detalles de implementación”? Justifique.

40. ¿Cuáles son los dos aspectos que hacen importante considerar a la modularidad?

41. ¿Para qué se utiliza la jerarquía?

42. ¿Cómo denominamos a la caracterización precisa de propiedades estructurales o de comportamiento que comparten una serie de entidades?

43. ¿Las clases implementan tipos?

44. ¿Los tipos implementan clases?

45. ¿Cómo denominamos a los lenguajes que hacen una comprobación de tipos estricta?

46. ¿Cómo denominamos a los lenguajes que no hacen una comprobación de tipos estricta?

47. ¿A qué se denomina ligadura estática (temprana)?

48. ¿A qué se denomina ligadura dinámica (tardía)?

49. ¿Es lo mismo la comprobación estricta de tipos y la ligadura dinámica?

50. ¿Cómo se denomina la característica que permite a diferentes objetos actuar al mismo tiempo?

51. ¿A qué se denomina concurrencia pesada?

52. ¿A qué se denomina concurrencia ligera o liviana?

53. ¿La concurrencia es la propiedad que distingue un objeto activo de uno que no lo está?

54. ¿Cómo se denomina la característica en orientación a objetos que permite conservar el estado de un objeto en el tiempo y el espacio?

55. ¿Qué cosas se persisten?

56. Defina qué es un objeto desde la perspectiva de la cognición humana.

57. ¿Un objeto es real o abstracto? Justifique.

58. ¿Los objetos poseen límites físicos precisos o imprecisos?

59. ¿Cuáles son las tres cosas que debe tener un objeto?

60. ¿Cuál es la palabra que se puede utilizar como sinónimo de objeto?

61. ¿Cuál es la palabra que se puede utilizar como sinónimo de instancia?

62. ¿Cómo definiría el estado de un objeto?

63. ¿A qué definimos propiedad de un objeto?

64. ¿Qué es lo que distingue a “un objeto” de los “valores simples”?

65. ¿Cómo definiría el comportamiento de un objeto?

66. ¿El comportamiento de un objeto se ve afectado por el estado del mismo o bien que el comportamiento del objeto es función de su estado?

67. ¿Algunos comportamientos pueden alterar el estado de un objeto?

68. Se puede afirmar que el estado de un objeto termina siendo los resultados acumulados de su comportamiento.

69. ¿A qué definiría como operación (método/función miembro)?

70. ¿Cuáles son las tres operaciones más comunes?

71. ¿Cuáles son las dos operaciones habituales que se utilizan para crear y destruir instancias de clases?

72. ¿Qué tipo de operación es el modificador?

73. ¿Qué tipo de operación es el selector?

74. ¿Qué tipo de operación es el iterador?

75. ¿Qué tipo de operación es el constructor?

76. ¿Qué tipo de operación es el destructor?

77. ¿Cómo denominamos operaciones fuera de las clases en aquellos programas orientados a objetos que permiten colocarlas (ej. C++)?

78. ¿Todos los métodos son operaciones?

79. ¿Todas las operaciones son métodos?

80. Dado el protocolo de un objeto (todos sus métodos y subprogramas libres asociados al objeto si el lenguaje lo permite) es conveniente dividirlo en grupos lógicos más pequeños de comportamiento?¿Por qué?

81. ¿Cómo denominamos a los grupos lógicos más pequeños de comportamiento del protocolo total de un objeto?

82. ¿Cuáles son las dos responsabilidades más importantes que posee un objeto?

83. ¿Es relevante el orden en que se invocan las operaciones de un objeto?

84. ¿Por qué decimos que los objetos se pueden considerar como máquinas?

85. ¿Qué es la identidad de un objeto?

86. Dadas dos variable X e Y del mismo tipo ¿qué significa que ambas son iguales?

87. Dadas dos variable X e Y del mismo tipo ¿qué significa asignarle Y a X?

88. Dadas dos variable X e Y del mismo tipo ¿qué significa clonar X en Y?

89. ¿Qué significa realizar una clonación superficial?

90. ¿Qué significa realizar una clonación profunda?

91. ¿Qué es el ciclo de vida de un objeto?

92. ¿Cómo se libera el espacio ocupado por un objeto?

93. ¿Qué tipos de relaciones existen entre los objetos?

94. ¿Cómo podemos definir al enlace entre objetos?

95. ¿Cómo pueden ser los mensajes entre dos objetos en una relación de enlace?

96. ¿Qué es un mensaje unidireccional?

97. ¿Qué es un mensaje bidireccional?

98. ¿Quién inicia el paso de un mensaje entre dos objetos en una relación de enlace?

99. ¿Cuáles son los roles o papeles que puede desempeñar un objeto en una relación de enlace?

100. ¿Qué significa que un objeto actúe como “Actor”?

101. ¿Qué significa que un objeto actúe como “Servidor”?

102. ¿Qué significa que un objeto actúe como “Agente”?

103. Dados dos objetos A y B, si A le puede enviar un mensaje a B, estando ambos relacionados por enlace, decimos que B respecto de A está: ……

104. ¿Cuáles son las cuatro formas de visibilidad que puede poseer un objeto servidor respecto de un objeto cliente?

105. En una relación de enlace de dos objetos, cuando uno le pasa un mensaje al otro, además de adoptar roles ambos deben
estar:……..

106. ¿Cuáles son las posibles formas de sincronización?
107. ¿Qué significa que dados dos objetos A y B estos están secuencialmente sincronizados?
108. ¿Qué significa que la forma de sincronizarse de un conjunto de objetos es vigilada?
109. ¿Qué significa que la forma de sincronizarse de un conjunto de objetos es síncrona?
110. ¿El enlace es una relación de igual a igual o jerárquica?
111. ¿La agregación es una relación de igual a igual o jerárquica?
112. ¿Qué tipo de jerarquía denota la agregación?
113. ¿Qué otros nombres recibe el “todo” en una relación de agregación?
114. ¿En una relación de agregación las “partes” forman parte del estado del “todo”?
115. ¿Qué tipos de agregación existen?
116. ¿Qué caracteriza a la agregación con contención física?
117. ¿Qué es una clase?
118. ¿La interfaz de la clase proporciona su visión interna?
119. ¿La implementación de la clase proporciona su visión externa?
120. ¿En cuántas partes la podemos dividir una interfaz en términos de la accesibilidad o visibilidad que posee?
121. ¿Qué tipos básicos de relaciones existen entre las clases?
122. ¿Qué relaciones entre clases se desprenden de las tres relaciones básicas?
123. ¿La asociación denota una dependencia semántica y la dirección de esta asociación?
124. ¿Qué significa la cardinalidad en una relación?
125. ¿Qué cardinalidad puede existir entre clases relacionadas por asociación?
126. ¿Qué es la herencia?
127. ¿Cuántos tipos de herencia existen?
128. ¿A qué se denomina herencia simple?
129. ¿A qué se denomina herencia múltiple?

130. ¿Cómo se denomina a la clase que no se espera tener instancias de ella y solo se utilizará para heredar a otras clases?

131. ¿Cómo se denomina a la clase que se espera tener instancias de ella y puede utilizarse para heredar a otras clases o no?

132. ¿Cómo se denomina al método de una clase abstracta que no posee implementación y fue pensado para que sea implementado en las sub clases que lo heredan?

133. ¿Cómo se denomina a la clase más generalizada en una estructura de clases?

134. ¿Qué es el polimorfismo?

135. ¿Cómo se denomina cuando una clase posee métodos que comparten el nombre y se diferencias por su firma?

136. ¿Qué sentencias de código se evitan utilizar cuando se aplica correctamente el polimorfismo?

137. ¿Qué es la agregación cómo relación entre clases?

138. ¿Qué formas de contención física existen en la agregación?

139. ¿Qué características posee la contención física por valor?

140. ¿Qué características posee la contención física por referencia?

141. ¿Qué es una relación de uso?

142. ¿Qué es la instanciación?

143. ¿Todo objeto es una instancia de una clase?

144. ¿Qué es una metaclase?

145. ¿Qué métricas hay que observar para determinar la calidad de una abstracción?

146. ¿Qué es el acoplamiento?

147. ¿Qué es la cohesión?

148. ¿Qué es la suficiencia?

149. ¿Qué es la compleción?

150. ¿Qué significa ser primitivo?

151. ¿Qué se debe observar al momento de decidir si una abstracción debe implementar un determinado comportamiento o no?

152. ¿Qué formas puede adoptar el paso de un mensaje?

153. ¿Qué características posee un mensaje síncrono?

154. ¿Qué características posee un mensaje de abandono inmediato?

155. ¿Qué características posee un mensaje de intervalo?

156. ¿Qué características posee un mensaje Asíncrono?

157. ¿Qué significa que una abstracción está accesible a otra?

158. ¿Qué expresa la ley de Demeter?

159. ¿Cuál es la consecuencia inmediata de aplicar la ley de Demeter?

160. ¿Cuáles son las cuatro formas fundamentales por las cuales un objeto X puede hacerse visible a un objeto Y?

161. ¿Para qué sirve clasificar a los objetos?

162. ¿Por qué es tan difícil la clasificación de objetos?

163. ¿Cómo es el rol del observador en la clasificación de objetos?

164. ¿Cuáles son las aproximaciones generales a la clasificación?

165. ¿Qué es la categorización clásica?

166. ¿Qué es el agrupamiento conceptual?

167. ¿Qué es la teoría de prototipos?

168. ¿Qué es una abstracción clave?

169. ¿Qué son los mecanismos?

UNIDAD II

170. ¿Qué es un campo de una clase?

171. ¿Qué es un método de una clase?

172. ¿A qué denominamos sobrecarga?

173. ¿Qué es una propiedad de una clase?

174. ¿Qué tipos de propiedades existen?

175. ¿Qué ámbitos pueden tener los campos, métodos y propiedades de las clases?

176. ¿Qué características posee cada ámbito existente si se lo aplico a un campo, un método y una propiedad de una clase?

177. ¿Para qué se utilizan los constructores?

178. ¿A qué se denomina tiempo de vida de un objeto?

179. ¿Para administrar las instancias de .NET se utiliza un contador de referencias?

180. ¿Qué objeto es el encargado de liberar el espacio ocupado por objetos que ya no se utilizan?

181. ¿Qué son los sucesos?

182. ¿Qué se utiliza para declarar un suceso?

183. ¿Cómo se logra que ocurra un suceso?

184. ¿Cómo se pueden atrapar los sucesos?

185. ¿Cómo le indica a un evento que desea cambiar el tipo del argumento que por defeto es EventArgs?

186. ¿Cómo y qué cosas se pueden compartir en una clase?

187. ¿Qué características poseen los campos compartidos?

188. ¿Qué características poseen los métodos compartidos?

189. ¿Qué características poseen los sucesos compartidos?

190. ¿Qué son y para que se pueden utilizar las clases anidadas?

191. ¿Qué ámbitos / modificadores de acceso existen? Explique las característcas de cada uno.

192. ¿Qué cosas se pueden heredar?

193. ¿Cómo y para qué se puede aprovechar en la práctica el polimorfismo?

194. ¿Cómo y para qué se utiliza la clase derived?

195. ¿Qué representa this?
196. ¿Qué clase representa a la clase base?
197. ¿Para qué se usa una clase abstracta?
198. ¿Para qué se usa una clase sellada?
199. ¿Qué es la sobreescritura?
200. ¿Qué elementos se pueden sobreescribir?
201. ¿A qué se denomina sombreado de métodos?
202. ¿Qué característica peculiar posee el sombrado vs la sobre-escritura?

UNIDAD III

203. ¿Qué es un Framework?
204. ¿Qué son los frozen-spots en un Framework?
205. ¿Qué son los hot-spots en un Framework?
206. ¿Cómo se puede clasificar un Framework según su extensibilidad?
207. ¿Qué es un Framework de Caja Blanca?
208. ¿Qué es un Framework de Caja Negra?
209. ¿Qué ventajas posee utilizar un Framework?
210. ¿Qué problemas resuelve .NET Framework?
211. ¿Qué es y qué permite hacer el CLR?
212. ¿Qué es el MSIL?
213. ¿Qué es el CTS?
214. ¿Qué es el CLS?
215. ¿Qué es una excepción?
216. ¿Qué se coloca en el bloque “Catch”?

217. ¿Cómo construiría un objeto del tipo “Exception” personalizado?

218. ¿Qué ocurre si en el bloque de código donde se produce la excepción el error no está siendo tratado?

219. ¿Cuál es el objeto de mayor jerarquía para el manejo de excepciones?

220. ¿En qué namespace se encuentra la clase Exception?

221. ¿Cuáles son las dos clases genéricas más importantes definidas en el FrameWork además de Exception?

222. ¿Qué instrucción se utiliza para poner en práctica el control e interceptar las excepciones?

223. ¿Dónde se coloca el código protegido contra excepciones si se iniciara una excepción?

224. ¿Qué tipo de excepción se utiliza para interceptar un error de división por cero?

225. ¿Qué tipo de excepción se utiliza para interceptar una DLL que tiene problemas al ser cargada?

226. ¿Qué colocaría dentro de una clausula “Catch” para especificar una condición adicional que el bloque “Catch” deberá evaluar como verdadero para que sea seleccionada?

227. ¿Si se desea colocar código de limpieza y liberación de recursos para que se ejecute cuando una excepción se produzca, dónde lo colocaría?

228. ¿Qué instrucción se utiliza para provocar un error y que el mismo se adapte al mecanismo de control de excepciones?

229. ¿Escriba el código que permitiría provocar una excepción del tipo “ArgumentException”?

230. ¿Cómo construiría un objeto del tipo “Exception” personalizado?

231. ¿Cómo armaría un “Catch” personalizado para que se ejecute cuando se de la excepción “ClienteNoExisteException”?

232. ¿Dónde se encuentran las instancias de los objetos administrados por el GC?

233. ¿Cuáles son los dos métodos más notorios que deben implementar las clases para trabajar correctamente con la recolección de elementos no utilizados y matar las instancias administradas y no administradas?

234. ¿Qué método se utiliza para que el GC recolecte los elementos no utilizados?

235. ¿Qué método se utiliza para suspender el subproceso actual hasta que el subproceso que se está procesando en la cola de finalizadores vacíe dicha cola?

236. ¿Qué método se ejecuta en los objetos cuando se ejecuta el método collect del GC?

237. ¿Qué método debería exponer una clase bien diseñada teniendo en consideración que no posee destructor?

238. ¿Cómo obtengo el método “Dispose”?

239. ¿Qué se programa en el método “Dispose”?

240. ¿Se pueden combinar el uso de “Dispose” y el Destructor?

241. ¿A qué se denomina “Resurrección de Objetos”?

242. ¿A qué se denomina “Generación” en el contexto de la recolección de elementos no utilizados?

243. ¿Qué valores puede adoptar la “Generación” de un objeto?

244. ¿Cómo se puede obtener el número de veces que se ha producido la recolección de elementos no utilizados para la generación de objetos especificada?

245. ¿Cómo se obtiene el número de generación actual de un objeto?

246. ¿Cómo se puede recuperar el número de bytes que se considera que están asignados en la actualidad?

247. ¿Qué utiliza para convertir un objeto en “no” válido para la recolección de elementos no utilizados desde el principio de la rutina actual hasta el momento en que se llamó a este método?

248. ¿Cómo se solicita que el sistema no llame al finalizador del objeto especificado?

249. ¿Cómo se solicita que el sistema llame al finalizador del objeto especificado, para el que previamente se ha llamado a “SuppressFinalize”?

250. ¿Cómo se obtiene el número máximo de generaciones que el sistema admite en la actualidad?

UNIDAD IV

251. ¿Para qué se utilizan las interfaces?

252. ¿Cómo se implementa una interfaz?

253. ¿Se pueden heredar las interfaces?

254. ¿Se puede implementar un tipo de polimorfismo peculiar por medio de interfaces?
255. ¿Para qué se utiliza la interfaz IComparable?
256. ¿Para qué se utiliza la interfaz IComparer?
257. ¿Para qué se utiliza la interfaz ICloneable?
258. ¿Para qué se utiliza la interfaz IEnumerable?
259. ¿Para qué se utiliza la interfaz IEnumerator?
260. ¿Qué es un delegado?
261. ¿A qué elementos se les puede delegar?
262. ¿Qué se puede delegar?
263. ¿Cómo construiría un delegado?
264. ¿Cómo implementaría un procedimiento de devolución de llamadas?
265. ¿Para qué sirve la multidifusión de delegados?

UNIDAD V

266. ¿Qué son y para qué se usan los tipos genéricos?
267. ¿A partir de que versión de c# se pueden utilizar?
268. Enumere las ventajas de utilizar genéricos.
269. ¿A qué elementos se le pueden aplicar genericos?
270. ¿Cuáles son los usos más comunes de los genéricos?
271. ¿Qué aspectos trascendentes se deben considerar al crear clases genéricas?
272. ¿Cués es la diferencia entre heredar de una clase genérica abierta y una clase genérica cerrada?
273. ¿Por qué es importante colocar restricciones en los parámetros de tipo?
274. ¿Cuáles son los tipos de restricciones que se le pueden colocar a un parámetro de tipo?
275. ¿Ejemplifique cómo crearía un método genérico con un parámetro de tipo?

276. ¿Qué es LinQ?

277. ¿Qué orígenes de datos se pueden consultar con LinQ?

278. ¿Cuáles son las partes básicas de una consulta LinQ?

279. ¿Qué especifica la consulta en una estructura de LinQ?

280. Mencione al menos tres clausulas (las más importantes) que se usan en una expresión de consulta LinQ.

281. Explique que hace cada cláusula enumerada en la pregunta anterior

282. Dado que una expreseión de consulta genera un Ienumerable, enumere y explique los métodos de extensión que posee Ienumerable. (p.e Count)

283. ¿Qué utilizaría para ordenar en una expresión LinQ?

284. ¿Qué utilizaría para lograr una unión entre dos origenes de datos en una expresión LinQ?

285. ¿Cómo se pueden generar nuevos tipos utilizando LinQ?

286. ¿Qué es una expresión Lambda?

287. Ejemplifique cómo se puede utilizar una expresión lambda para realizar una consulta.

288. ¿Qué debo realizar para crear una expresión lambda?

289. ¿Indique como puede declarar un delegado utilizndo Func y que significa cada elemento utilizado?

290. Crear un delegado utilizando Func que posea tres parámetros de entrada y uno de salida. Los parámetros de entrada son: el primero int, el segundo double y el tercero bool. El parámetro de retorno es de tipo bool.

UNIDAD VI

291. ¿Qué características posee un esquema cliente - servidor?

292. ¿Qué significa pasar información batch?

293. ¿Qué significa pasar información on line?

294. ¿Qué es un protocolo?

295. ¿Qué protocolo usa una red de área local?
296. ¿Qué protocolo usa internet?
297. ¿Qué hace el protocolo IP?
298. ¿Qué ventajas tiene distribuir procesos?
299. ¿Qué ventajas tiene distribuir almacenamientos?
300. ¿Qué es un socket?
301. ¿Qué característica posee un socket sincrónico?
302. ¿Qué característica posee un socket asincrónico?
303. ¿Qué objeto se puede utilizar para construir un navegador?
304. ¿Para qué se utilizan los puertos de la pc?
305. ¿Qué puertos posee una PC?
306. ¿Cómo funciona el puerto paralelo?
307. ¿Cómo funciona el puerto serie?
308. ¿Cómo funciona el puerto USB?
309. ¿Qué es la domótica?

Resolución de Preguntas Conceptuales

UNIDAD I

1. Un software de gran magnitud es complejo principalmente por: cantidad de elementos (posee muchos componentes, módulos y objetos); interacciones entre partes (los elementos se comunican y dependen entre sí); dificultad de comprensión global (no es posible entender todo el sistema al mismo tiempo); evolución constante (el software cambia con el tiempo, agregando funcionalidades y corrigiendo errores); y limitaciones cognitivas humanas (la capacidad humana para razonar sobre muchos detalles simultáneamente es limitada).

2. Los cinco atributos de un sistema complejo son:

1) Jerarquía: el sistema está organizado en niveles o subsistemas.

2) Descomposición: el sistema puede dividirse en partes más pequeñas.

3) Abstracción: cada nivel oculta detalles innecesarios para simplificar la comprensión.

4) Interrelación entre componentes: las partes interactúan entre sí.

5) Evolución: el sistema cambia y crece a lo largo del tiempo.

3. Las dos jerarquías más importantes en la orientación a objetos para sistemas complejos son: la jerarquía de generalización/especialización (es-un), que representa relaciones de herencia entre clases; y la jerarquía de agregación/composición (parte-de), que representa relaciones todo-parte entre objetos.

4. La complejidad se puede enfrentar mediante: descomposición (dividir el sistema en partes más pequeñas y manejables), abstracción (centrarse en los aspectos esenciales y ocultar detalles innecesarios) y jerarquía (organizar los elementos en niveles para facilitar su comprensión). Estas herramientas permiten trabajar sobre porciones reducidas del problema sin perder la visión del conjunto.

5. Las dos formas de descomposición más conocidas son la descomposición algorítmica y la descomposición orientada a objetos.

6. La descomposición algorítmica divide el sistema en procedimientos o funciones que ejecutan una secuencia de pasos; el foco está en qué pasos se realizan. La descomposición orientada a objetos divide el sistema en objetos que encapsulan estado y comportamiento y colaboran entre sí mediante mensajes; el foco está en quién es responsable de cada acción.

7. La abstracción permite identificar y modelar los aspectos esenciales de un objeto, ignorando los detalles irrelevantes para el problema que se está resolviendo. En orientación a objetos, la abstracción define qué es y qué hace un objeto, establece límites conceptuales claros y facilita la comprensión y el manejo de la complejidad.

8. La jerarquía organiza las abstracciones en niveles, permitiendo estructurar el sistema, reutilizar comportamiento y expresar relaciones es-un (herencia) y parte-de (agregación).

9. El diseño orientado a objetos es un desarrollo evolutivo, no revolucionario. Justificación: integra conceptos previos como modularidad, ocultamiento de información y tipos abstractos de datos; no descarta paradigmas anteriores, sino que los extiende y organiza dentro de un modelo más poderoso para manejar la complejidad.

10. Se manejan dos modelos básicos: 1) Modelo lógico: describe clases, objetos y sus relaciones. 2) Modelo físico: describe módulos, procesos y la implementación del sistema. Estos modelos permiten separar el diseño conceptual de la implementación concreta.

11. La programación orientada a objetos es un paradigma de programación que organiza el software como un conjunto de objetos que encapsulan estado, comportamiento e identidad, y que interactúan entre sí mediante mensajes.

12. El diseño orientado a objetos es el proceso mediante el cual se definen las clases, responsabilidades y relaciones necesarias para implementar una solución orientada a objetos. Su objetivo es transformar el análisis del problema en una estructura implementable.

13. El análisis orientado a objetos es la etapa en la que se estudia el dominio del problema, identificando los objetos, conceptos y relaciones relevantes, sin considerar aún detalles de implementación.

14. Los elementos fundamentales del modelo de objetos son: Abstracción, Encapsulamiento, Modularidad y Jerarquía. Estos elementos constituyen la base del paradigma orientado a objetos.

15. Los elementos secundarios del modelo de objetos son: Tipificación, Concurrencia y Persistencia. Complementan a los fundamentales y amplían el poder expresivo del modelo.

16. La abstracción consiste en destacar las características esenciales de un objeto, omitiendo detalles innecesarios, para representar el problema de manera clara y manejable.

17. El encapsulamiento consiste en ocultar la implementación interna de un objeto y exponer solo su interfaz, protegiendo la integridad del objeto y reduciendo dependencias.

18. La modularidad consiste en dividir el sistema en módulos independientes y cohesivos, permitiendo desarrollar, modificar y mantener cada parte sin afectar al resto.

19. La jerarquía organiza las abstracciones en niveles, estableciendo relaciones de generalización y composición que facilitan la comprensión y reutilización del software.

20. La tipificación permite definir y restringir el uso de los objetos mediante tipos, asegurando coherencia y consistencia en el diseño y la implementación.

21. La concurrencia es la capacidad de un sistema de manejar múltiples actividades en forma simultánea o pseudo-simultánea, mediante objetos activos y pasivos.

22. La persistencia es la capacidad de los objetos de conservar su estado más allá de la ejecución del programa, permitiendo que los datos sobrevivan en el tiempo.

23. Las características esenciales de un objeto que lo distinguen de todos los demás tipos de objetos y que proporcionan fronteras conceptuales nítidamente definidas respecto a la perspectiva del observador se denotan mediante su abstracción.

24. Un objeto cliente es aquel que envía mensajes solicitando la ejecución de operaciones a otro objeto.

25. Un objeto servidor es aquel que recibe mensajes de otros objetos y ejecuta las operaciones solicitadas, brindando servicios.

26. Meyer denomina modelo contractual de programación a la concepción según la cual los objetos interactúan a través de contratos formados por precondiciones, postcondiciones e invariantes.

27. Un contrato entre objetos establece las obligaciones y garantías que deben cumplirse durante la interacción entre ellos.

28. Las formas en que un objeto puede actuar y/o reaccionar, constituyendo su visión externa completa,

estática y dinámica, se denominan interfaz.

29. El conjunto completo de operaciones que puede realizar un cliente sobre un objeto, junto con las formas de invocación que admite, se denomina protocolo.

30. Cuando se dice que un concepto central de la abstracción es el de invariancia, se hace referencia a aquellas condiciones que deben mantenerse verdaderas durante toda la vida del objeto, independientemente de su estado.

31. Para cualquier operación asociada a un objeto se deben definir: precondiciones, postcondiciones y excepciones posibles.

32. Una precondición es una condición que debe cumplirse antes de que una operación sea ejecutada correctamente.

33. Una postcondición es una condición que debe cumplirse después de que una operación haya sido ejecutada.

34. Una excepción es una situación anómala que impide que una operación cumpla con su postcondición.

35. Un mensaje es una solicitud que un objeto envía a otro para que ejecute una operación determinada.

36. Sí, el encapsulamiento es un concepto complementario a la abstracción, ya que mientras la abstracción define qué es un objeto, el encapsulamiento define cómo se accede a él y protege su implementación.

37. El elemento de un objeto que captura su vista externa se denomina interfaz.

38. El elemento de un objeto que captura su vista interna, incluyendo los mecanismos que consiguen el comportamiento deseado, se denomina implementación.

39. El concepto de 'ocultar los detalles de implementación' se asocia más correctamente a evitar el uso inapropiado de los detalles de implementación, y no simplemente a esconderlos. La idea no es solo ocultarlos sino evitar que sean utilizados de manera incorrecta.

40. Los dos aspectos que hacen importante considerar la modularidad son: la reducción de la

complejidad y la facilidad de mantenimiento y evolución del sistema.

41. La jerarquía se utiliza para organizar y clasificar abstracciones, facilitando la comprensión y reutilización del sistema.

42. La caracterización precisa de propiedades estructurales o de comportamiento que comparten una serie de entidades se denomina tipo.

43. Sí, las clases implementan tipos.

44. No, los tipos no implementan clases. El tipo es un concepto más abstracto que puede existir independientemente de la clase.

45. Los lenguajes que hacen una comprobación estricta de tipos se denominan lenguajes fuertemente tipados.

46. Los lenguajes que no hacen una comprobación estricta de tipos se denominan lenguajes débilmente tipados.

47. La ligadura estática o temprana es aquella en la que la asociación entre una llamada y su implementación se resuelve en tiempo de compilación.

48. La ligadura dinámica o tardía es aquella en la que la asociación entre una llamada y su implementación se resuelve en tiempo de ejecución.

49. No, la comprobación estricta de tipos y la ligadura dinámica no son lo mismo. Son conceptos distintos e independientes: la comprobación de tipos refiere a cuándo se validan los tipos, mientras que la ligadura refiere a cuándo se resuelve qué método ejecutar.

50. La característica que permite a diferentes objetos actuar al mismo tiempo se denomina concurrencia.

51. La concurrencia pesada es aquella que se basa en procesos independientes del sistema operativo.

52. La concurrencia ligera o liviana es aquella que se basa en hilos de ejecución dentro de un mismo proceso.

53. Sí, la concurrencia es la propiedad que distingue un objeto activo de uno que no lo está.

54. La característica que permite conservar el estado de un objeto en el tiempo y el espacio se denomina persistencia.

55. Se persisten los estados y los datos de los objetos, es decir, los valores de sus atributos en un momento dado.

56. Un objeto, desde la perspectiva de la cognición humana, es una entidad identificable que posee significado propio dentro de un determinado contexto.

57. Un objeto puede ser real o abstracto. Es real cuando representa entidades físicas del mundo real y es abstracto cuando representa conceptos, ideas o roles del dominio del problema.

58. Los objetos poseen límites conceptuales precisos, aunque no necesariamente límites físicos precisos.

59. Las tres cosas que debe tener un objeto son: estado, comportamiento e identidad.

60. La palabra que se puede utilizar como sinónimo de objeto es entidad.

61. La palabra que se puede utilizar como sinónimo de instancia es objeto.

62. El estado de un objeto es el conjunto de valores de sus propiedades en un momento determinado.

63. Una propiedad de un objeto es una característica que describe su estado.

64. Lo que distingue a un objeto de los valores simples es que el objeto posee identidad propia, mientras que los valores simples no tienen identidad; dos valores iguales son indistinguibles.

65. El comportamiento de un objeto es el conjunto de operaciones que puede realizar en respuesta a los mensajes que recibe.

66. El comportamiento de un objeto es función de su estado, ya que el resultado de una operación puede variar según los valores actuales de sus propiedades.

67. Sí, algunos comportamientos pueden alterar el estado de un objeto (operaciones modificadoras), mientras que otros solo lo consultan.

68. Sí, se puede afirmar que el estado de un objeto termina siendo el resultado acumulado de su

comportamiento, ya que cada operación modificadora que se ejecuta va dejando su huella en el estado.

69. Una operación (método o función miembro) es un servicio que un objeto ofrece a otros objetos, es decir, una acción que puede ejecutar cuando recibe un mensaje.

70. Las tres operaciones más comunes son: selector, modificador e iterador.

71. Las dos operaciones habituales que se utilizan para crear y destruir instancias de clases son: constructor y destructor.

72. El modificador es una operación que cambia el estado de un objeto.

73. El selector es una operación que consulta el estado de un objeto sin modificarlo.

74. El iterador es una operación que permite recorrer una colección de elementos de manera ordenada.

75. El constructor es una operación especial que crea e inicializa una instancia de una clase.

76. El destructor es una operación que libera los recursos asociados a un objeto cuando finaliza su ciclo de vida.

77. Las operaciones fuera de las clases, en aquellos programas orientados a objetos que permiten colocarlas (ej. C++), se denominan subprogramas libres.

78. Sí, todos los métodos son operaciones.

79. No, no todas las operaciones son métodos, ya que algunos lenguajes permiten operaciones (subprogramas libres) que no pertenecen a ninguna clase.

80. Sí, es conveniente dividir el protocolo de un objeto en grupos lógicos más pequeños de comportamiento, porque mejora la comprensión, la cohesión y la mantenibilidad del diseño.

81. Los grupos lógicos más pequeños de comportamiento del protocolo total de un objeto se denominan roles.

82. Las dos responsabilidades más importantes que posee un objeto son: mantener su estado y ofrecer servicios a otros objetos.

83. Sí, es relevante el orden en que se invocan las operaciones de un objeto, ya que el estado actual determina el resultado de las operaciones y el estado resultante.

84. Decimos que los objetos se pueden considerar como máquinas porque reciben mensajes como entrada y producen resultados como salida, de manera análoga a una máquina que recibe entradas y produce salidas.

85. La identidad de un objeto es la característica que lo distingue de cualquier otro, incluso si posee el mismo estado y comportamiento.

86. Dadas dos variables X e Y del mismo tipo, decir que son iguales significa que sus valores son equivalentes, es decir, que sus estados son idénticos.

87. Asignarle Y a X significa que ambas referencias apuntan al mismo objeto en memoria; no se crea una copia.

88. Clonar X en Y significa crear un nuevo objeto Y con el mismo estado que X, pero con identidad propia.

89. La clonación superficial consiste en copiar el objeto, pero compartiendo las referencias internas (los objetos referenciados no se duplican).

90. La clonación profunda consiste en copiar el objeto y todos los objetos que este referencia, creando objetos completamente independientes.

91. El ciclo de vida de un objeto comprende desde su creación (mediante el constructor) hasta su destrucción (mediante el destructor o el recolector de basura).

92. El espacio ocupado por un objeto se libera mediante el mecanismo de destrucción, ya sea explícito (destructor llamado por el programador) o automático a través del recolector de basura, según el lenguaje.

93. Los tipos de relaciones que existen entre los objetos son: enlace y agregación.

94. El enlace entre objetos se define como una conexión física o conceptual que permite el intercambio de mensajes entre dos objetos.

95. Los mensajes entre dos objetos en una relación de enlace pueden ser unidireccionales o

bidireccionales.

96. Un mensaje unidireccional es aquel en el que solo un objeto envía mensajes al otro, y el receptor no envía mensajes al emisor.

97. Un mensaje bidireccional es aquel en el que ambos objetos pueden enviarse mensajes mutuamente en cualquier dirección.

98. El paso de un mensaje entre dos objetos en una relación de enlace lo inicia el objeto que actúa como cliente (actor).

99. Los roles o papeles que puede desempeñar un objeto en una relación de enlace son: Actor, Servidor y Agente.

100. Un objeto actúa como Actor cuando solo inicia acciones enviando mensajes a otros objetos, sin ser destinatario de mensajes de otros.

101. Un objeto actúa como Servidor cuando solo recibe mensajes y ejecuta operaciones solicitadas por otros objetos, sin enviar mensajes por iniciativa propia.

102. Un objeto actúa como Agente cuando cumple ambos roles simultáneamente: recibe mensajes de otros objetos (actúa como servidor) y a su vez envía mensajes a otros objetos (actúa como cliente). Es un intermediario entre otros objetos.

103. Dados dos objetos A y B, si A le puede enviar un mensaje a B, estando ambos relacionados por enlace, decimos que B respecto de A está visible.

104. Las cuatro formas de visibilidad que puede poseer un objeto servidor respecto de un objeto cliente son: como atributo del objeto cliente, como parámetro de una operación del cliente, como variable local dentro de una operación del cliente, y como variable global accesible al cliente.

105. En una relación de enlace de dos objetos, cuando uno le pasa un mensaje al otro, además de adoptar roles ambos deben estar sincronizados.

106. Las posibles formas de sincronización son: secuencial, vigilada (guarded) y síncrona.

107. La sincronización secuencial significa que la semántica del servidor se garantiza solo si los clientes

acceden de forma ordenada y sin concurrencia, es decir, un cliente a la vez.

108. La sincronización vigilada (guarded) significa que varios objetos pueden intentar acceder simultáneamente al servidor, pero un mecanismo de control garantiza que solo uno sea atendido a la vez, haciendo esperar al resto.

109. La sincronización síncrona significa que el objeto cliente espera bloqueado hasta que el objeto servidor esté disponible y pueda recibir el mensaje, asegurando la comunicación entre ambos.

110. El enlace es una relación de igual a igual (peer-to-peer), no jerárquica.

111. La agregación es una relación jerárquica, ya que establece una relación todo-parte entre objetos.

112. La agregación denota una jerarquía de composición (parte-de), en contraposición a la jerarquía de generalización/especialización (es-un) que denota la herencia.

113. El 'todo' en una relación de agregación recibe también los nombres de: contenedor, agregado o ensamblaje.

114. Sí, en una relación de agregación las 'partes' forman parte del estado del 'todo', ya que las propiedades del todo incluyen las referencias a sus partes.

115. Los tipos de agregación existentes son: agregación sin contención física (las partes existen independientemente) y agregación con contención física (el todo contiene físicamente a las partes, por valor o por referencia).

116. La agregación con contención física se caracteriza porque el objeto todo contiene directamente a sus partes; si la contención es por valor, las partes son destruidas junto con el todo; si es por referencia, las partes tienen existencia propia pero están fuertemente ligadas al todo.

117. Una clase es una descripción de un conjunto de objetos que comparten la misma estructura (atributos), el mismo comportamiento (métodos) y las mismas relaciones. Es una plantilla a partir de la cual se crean instancias (objetos).

118. No, la interfaz de la clase proporciona su visión externa (lo que el cliente puede ver y usar), no su visión interna.

119. No, la implementación de la clase proporciona su visión interna (cómo está construida y cómo funciona por dentro), no su visión externa.

120. Una interfaz se puede dividir en cuatro partes según su accesibilidad: pública (accesible para todos), protegida (accesible para la propia clase y sus subclases), privada (accesible solo dentro de la propia clase) y de paquete o interna (accesible dentro del mismo módulo o ensamblado).

121. Los tipos básicos de relaciones entre clases son tres: asociación, herencia y agregación.

122. De las tres relaciones básicas se desprenden: asociación (y sus variantes como dependencia y uso), herencia (simple y múltiple) y agregación (con sus variantes de composición y contención).

123. Sí, la asociación denota una dependencia semántica entre clases y puede tener dirección, lo que indica que una clase conoce a la otra pero no necesariamente a la inversa.

124. La cardinalidad en una relación expresa la cantidad de instancias de una clase que pueden estar relacionadas con instancias de otra clase.

125. La cardinalidad entre clases relacionadas por asociación puede ser: uno a uno (1:1), uno a muchos (1:N), muchos a uno (N:1) y muchos a muchos (N:M).

126. La herencia es una relación entre clases en la que una clase (subclase o clase derivada) adquiere la estructura y el comportamiento de otra clase (superclase o clase base), pudiendo agregar o redefinir comportamiento.

127. Existen dos tipos de herencia: la herencia simple y la herencia múltiple.

128. La herencia simple es aquella en la que una clase derivada hereda de una única clase base.

129. La herencia múltiple es aquella en la que una clase derivada hereda de más de una clase base al mismo tiempo.

130. La clase de la que no se espera tener instancias y que solo se utilizará para heredar a otras clases se denomina clase abstracta.

131. La clase de la que se espera tener instancias y que puede utilizarse para heredar a otras clases o no, se denomina clase concreta.

132. El método de una clase abstracta que no posee implementación y fue pensado para que sea implementado en las subclases que lo heredan se denomina método abstracto (o método virtual puro en C++).

133. La clase más generalizada en una estructura de clases se denomina clase raíz o clase base.

134. El polimorfismo es la capacidad de distintos objetos de responder de manera diferente al mismo mensaje, permitiendo que objetos de distintas clases sean tratados a través de un tipo común.

135. Cuando una clase posee métodos que comparten el nombre pero se diferencian por su firma (número o tipo de parámetros), se denomina sobrecarga (overloading).

136. Cuando se aplica correctamente el polimorfismo se evitan las sentencias de control de flujo condicional, principalmente las sentencias if-else encadenadas y las sentencias switch/case que verifican el tipo del objeto.

137. La agregación como relación entre clases representa una relación todo-parte, donde una clase (el todo) está compuesta de instancias de otra clase (las partes). Las partes contribuyen al estado del todo.

138. Las formas de contención física en la agregación son: contención por valor y contención por referencia.

139. La contención física por valor se caracteriza porque las partes están embebidas directamente en el objeto todo; cuando el todo se destruye, las partes también se destruyen automáticamente.

140. La contención física por referencia se caracteriza porque el objeto todo contiene un puntero o referencia a las partes; las partes pueden tener existencia independiente y su ciclo de vida no necesariamente está atado al del todo.

141. Una relación de uso es aquella en la que una clase utiliza los servicios de otra sin que exista una relación de herencia ni de agregación. Es una dependencia débil y generalmente temporal.

142. La instanciación es el proceso de crear un objeto concreto (instancia) a partir de una clase.

143. Sí, todo objeto es una instancia de una clase. Un objeto siempre pertenece a alguna clase que define su estructura y comportamiento.

144. Una metaclase es una clase cuyas instancias son a su vez clases. Es decir, es una clase de clases. Se utiliza en lenguajes que tratan las clases como objetos de primera clase.

145. Las métricas para determinar la calidad de una abstracción son: acoplamiento, cohesión, suficiencia, compleción y primitividad (ser primitivo).

146. El acoplamiento mide el grado de dependencia que existe entre distintas clases o módulos. Un bajo acoplamiento es deseable porque reduce el impacto de los cambios en otras partes del sistema.

147. La cohesión mide el grado de relación interna entre los elementos de una clase. Una alta cohesión es deseable porque indica que la clase tiene responsabilidades bien definidas y relacionadas entre sí.

148. La suficiencia indica que una clase posee al menos las operaciones mínimas necesarias para cumplir su responsabilidad, es decir, que tiene todo lo necesario para ser útil.

149. La compleción indica que una clase captura todas las características relevantes de la abstracción que representa, sin que falten comportamientos importantes.

150. Ser primitivo significa que una operación solo puede implementarse eficientemente con acceso directo a la representación interna del objeto; si puede implementarse externamente sin pérdida de eficiencia, no es primitiva.

151. Al momento de decidir si una abstracción debe implementar un determinado comportamiento, se debe observar si esa operación es primitiva (si requiere acceso interno) y si corresponde a las responsabilidades propias de la clase según su propósito.

152. El paso de un mensaje puede adoptar las siguientes formas: síncrono, de abandono inmediato, de intervalo y asíncrono.

153. Un mensaje síncrono se caracteriza porque el emisor espera bloqueado a que el receptor acepte el mensaje y complete su procesamiento antes de continuar.

154. Un mensaje de abandono inmediato se caracteriza porque el emisor intenta comunicarse con el receptor pero, si no está disponible de inmediato, abandona el intento sin esperar.

155. Un mensaje de intervalo se caracteriza porque el emisor espera al receptor solo durante un tiempo máximo determinado; si el receptor no responde en ese lapso, el emisor abandona la espera.

156. Un mensaje asíncrono se caracteriza porque el emisor envía el mensaje y continúa su ejecución sin esperar que el receptor lo procese; no hay bloqueo del emisor.

157. Que una abstracción esté accesible a otra significa que existe al menos un camino de visibilidad entre ellas que permite que una le envíe mensajes a la otra.

158. La ley de Demeter establece que un método de un objeto solo debería enviar mensajes a: el propio objeto (this), los objetos que son parámetros del método, los objetos que el método crea, y los objetos que son atributos del objeto.

159. La consecuencia inmediata de aplicar la ley de Demeter es que se reduce el acoplamiento entre clases, ya que los objetos no necesitan conocer la estructura interna de los objetos con los que interactúan.

160. Las cuatro formas fundamentales por las cuales un objeto X puede hacerse visible a un objeto Y son: X es atributo de Y, X es parámetro de un método de Y, X es creado localmente dentro de un método de Y, y X es una variable global accesible desde Y.

161. Clasificar a los objetos sirve para reducir la complejidad, organizar el conocimiento, facilitar la reutilización y establecer un vocabulario común entre los participantes del desarrollo.

162. La clasificación de objetos es tan difícil porque no existe una única clasificación correcta; depende del punto de vista del observador, del dominio del problema y del nivel de abstracción elegido.

163. El rol del observador en la clasificación de objetos es fundamental: la clasificación no es objetiva ni universal, sino que depende de la perspectiva y los objetivos del observador, lo que hace que distintos diseñadores puedan llegar a clasificaciones diferentes.

164. Las aproximaciones generales a la clasificación son: la categorización clásica, el agrupamiento conceptual y la teoría de prototipos.

165. La categorización clásica sostiene que todos los objetos con las mismas propiedades pertenecen a la misma categoría y que cada categoría puede definirse por un conjunto de propiedades necesarias y suficientes.

166. El agrupamiento conceptual sostiene que los objetos se agrupan según conceptos o descripciones que los caracterizan, y que primero se forma la descripción del grupo y luego se clasifica a los objetos según cuán bien se ajusten a ella.

167. La teoría de prototipos sostiene que algunas instancias son mejores representantes de una categoría que otras (prototipos), y que la pertenencia a una categoría se determina por el grado de similaridad con el prototipo.

168. Una abstracción clave es una clase u objeto que forma parte del vocabulario del dominio del problema; son las abstracciones que emergen naturalmente del análisis del dominio y son esenciales para la solución.

169. Los mecanismos son estructuras o patrones de colaboración entre objetos que resuelven problemas comunes del dominio. Son abstracciones del comportamiento colectivo de un conjunto de objetos que cooperan.

UNIDAD II

170. Un campo de una clase es una variable que pertenece a la clase o a sus instancias, y que almacena un valor que contribuye al estado del objeto o de la clase.

171. Un método de una clase es una función o procedimiento definido dentro de la clase que implementa el comportamiento de sus instancias o de la propia clase.

172. La sobrecarga (overloading) es la capacidad de definir dentro de una clase múltiples métodos con el mismo nombre pero diferente firma (distinto número o tipo de parámetros).

173. Una propiedad de una clase es un mecanismo que permite acceder y modificar los campos de un objeto de manera controlada, exponiendo una interfaz pública que encapsula el acceso al estado interno.

174. Los tipos de propiedades existentes son: de solo lectura (solo get), de solo escritura (solo set) y de lectura y escritura (get y set).

175. Los ámbitos que pueden tener los campos, métodos y propiedades de las clases son: público (public), privado (private), protegido (protected), interno (internal) y protegido interno (protected internal).

176. Public: accesible desde cualquier parte. Private: accesible solo dentro de la propia clase. Protected: accesible dentro de la clase y sus subclases. Internal: accesible dentro del mismo ensamblado. Protected Internal: accesible desde subclases o desde el mismo ensamblado.

177. Los constructores se utilizan para inicializar el estado de un objeto al momento de su creación, asegurando que el objeto quede en un estado válido desde el principio.

178. El tiempo de vida de un objeto es el período que transcurre desde su creación (instanciación) hasta su destrucción (liberación de memoria).

179. No, .NET no utiliza un contador de referencias para administrar las instancias. Utiliza un Garbage Collector (recolector de basura) que detecta los objetos inalcanzables y libera su memoria.

180. El objeto encargado de liberar el espacio ocupado por objetos que ya no se utilizan es el Garbage Collector (GC).

181. Los sucesos (eventos) son notificaciones que un objeto puede generar para informar a otros objetos que algo de interés ha ocurrido. Permiten implementar el patrón de diseño observador.

182. Para declarar un suceso se utiliza la palabra clave event junto con un tipo delegado que define la firma del manejador del evento.

183. Un suceso ocurre cuando el objeto que lo define lo invoca (lo lanza), generalmente mediante una llamada al delegado asociado al evento.

184. Los sucesos se pueden atrapar suscribiendo un manejador (handler) al evento mediante el operador +=. El método suscripto se ejecutará cuando el evento sea lanzado.

185. Para cambiar el tipo del argumento que por defecto es EventArgs, se define una clase propia que herede de EventArgs y se declara el delegado del evento con ese nuevo tipo como segundo parámetro.

186. En una clase se pueden compartir campos, métodos, propiedades y sucesos utilizando la palabra clave Shared (VB) o static (C#), lo que los hace pertenecientes a la clase y no a las instancias.

187. Los campos compartidos pertenecen a la clase y no a las instancias; existe una única copia del valor compartida por todos los objetos de la clase.

188. Los métodos compartidos pertenecen a la clase y pueden invocarse sin necesidad de crear una instancia. No pueden acceder a miembros de instancia directamente.

189. Los sucesos compartidos pertenecen a la clase y pueden ser lanzados o suscritos sin necesidad de

una instancia concreta del objeto.

190. Las clases anidadas son clases definidas dentro de otra clase. Se utilizan para encapsular lógica auxiliar que solo tiene sentido en el contexto de la clase contenedora, mejorando la organización y el encapsulamiento.

191. Los modificadores de acceso son: Public (accesible desde cualquier lugar), Private (solo dentro de la clase), Protected (en la clase y subclases), Internal (dentro del ensamblado), Protected Internal (subclases o el mismo ensamblado) y Private Protected (subclases dentro del mismo ensamblado).

192. Se pueden heredar los campos públicos y protegidos, los métodos públicos y protegidos, las propiedades públicas y protegidas y los constructores (con sintaxis especial). No se heredan los miembros privados ni los constructores directamente.

193. El polimorfismo se aprovecha en la práctica declarando variables o parámetros del tipo de la clase base o interfaz, y asignando objetos de subclases. Así, el mismo código puede operar sobre distintos tipos de objetos sin necesidad de conocer su tipo concreto.

194. La palabra clave base (C#) o MyBase (VB) permite acceder a la clase base desde la clase derivada, y se utiliza para invocar el constructor de la clase base o para llamar a métodos de la superclase que han sido sobreescritos.

195. this (C#) o Me (VB) representa la instancia actual de la clase, es decir, el objeto sobre el que se está ejecutando el método en curso.

196. La clase base se representa mediante la palabra clave base en C# o MyBase en VB.NET.

197. Una clase abstracta se usa cuando se quiere definir una clase que no puede ser instanciada directamente, pero que sirve como base para otras clases, estableciendo comportamiento común y métodos abstractos que las subclases deben implementar.

198. Una clase sellada (sealed en C# / NotInheritable en VB) se usa cuando se quiere impedir que otras clases hereden de ella, protegiendo su implementación y garantizando su comportamiento.

199. La sobreescritura (overriding) es el mecanismo por el cual una clase derivada redefine la implementación de un método heredado de la clase base, conservando la misma firma.

200. Los elementos que se pueden sobreescribir son los métodos virtuales (virtual en C# / Overridable en

VB), los métodos abstractos y las propiedades virtuales o abstractas.

201. El sombreado de métodos (shadowing) es cuando una clase derivada define un miembro con el mismo nombre que uno de la clase base, ocultándolo sin sobreescribirlo formalmente. Se utiliza la palabra clave new en C# o Shadows en VB.

202. La característica peculiar del sombreado vs la sobreescritura es que en el sombreado, si se accede al objeto a través de una referencia del tipo base, se ejecuta el método de la clase base (no el de la derivada). En la sobreescritura, el método de la derivada se ejecuta siempre gracias al enlace dinámico.

UNIDAD III

203. Un Framework es un marco de trabajo reutilizable que provee una arquitectura y un conjunto de componentes predefinidos para el desarrollo de aplicaciones en un dominio específico. Invierte el control del flujo respecto a una biblioteca tradicional.

204. Los frozen-spots (puntos congelados) en un Framework son las partes fijas e invariables que el framework provee y que no deben ser modificadas por el programador que lo utiliza.

205. Los hot-spots (puntos calientes) en un Framework son las partes diseñadas para ser extendidas o personalizadas por el programador, donde se implementa la lógica específica de la aplicación.

206. Un Framework se puede clasificar según su extensibilidad en: Framework de Caja Blanca (white-box) y Framework de Caja Negra (black-box).

207. Un Framework de Caja Blanca se extiende mediante herencia y sobreescritura de métodos. El programador necesita conocer la estructura interna del framework para poder personalizarlo.

208. Un Framework de Caja Negra se extiende mediante configuración y composición, generalmente a través de interfaces o delegados. El programador no necesita conocer la estructura interna.

209. Las ventajas de utilizar un Framework son: reutilización de código y diseño, reducción del tiempo de desarrollo, estandarización de soluciones, mayor calidad y menor cantidad de errores, y aprovechamiento de la experiencia acumulada en el dominio.

210. .NET Framework resuelve problemas de: portabilidad entre lenguajes, administración automática de

memoria, seguridad de tipos, manejo de excepciones, interoperabilidad entre lenguajes y despliegue simplificado de aplicaciones.

211. El CLR (Common Language Runtime) es el entorno de ejecución del .NET Framework. Permite ejecutar código administrado, gestiona la memoria automáticamente (Garbage Collector), controla la seguridad y garantiza la interoperabilidad entre lenguajes.

212. El MSIL (Microsoft Intermediate Language), también llamado IL o CIL, es el lenguaje intermedio al que se compilan todos los lenguajes .NET. Es independiente del procesador y es traducido a código nativo por el compilador JIT en tiempo de ejecución.

213. El CTS (Common Type System) es el sistema de tipos común de .NET que define cómo se declaran, usan y administran los tipos en el CLR, garantizando la interoperabilidad entre distintos lenguajes .NET.

214. El CLS (Common Language Specification) es un conjunto de reglas y restricciones que los lenguajes .NET deben cumplir para garantizar la interoperabilidad. Un código que respeta el CLS puede ser utilizado desde cualquier lenguaje .NET.

215. Una excepción es un objeto que representa una situación anómala o un error que ocurre durante la ejecución de un programa, interrumpiendo el flujo normal de ejecución.

216. En el bloque Catch se coloca el código que maneja la excepción: el tratamiento del error, mensajes al usuario o acciones de recuperación. Se especifica el tipo de excepción que se desea capturar.

217. Para construir un objeto del tipo Exception personalizado se debe crear una clase que herede de Exception (o de alguna subclase de Exception) e implementar los constructores necesarios, incluyendo el que acepta un mensaje descriptivo del error.

218. Si en el bloque de código donde se produce la excepción el error no está siendo tratado, la excepción se propaga hacia el método llamante. Si ningún método en la pila de llamadas la captura, el programa termina con un error en tiempo de ejecución.

219. El objeto de mayor jerarquía para el manejo de excepciones en .NET es System.Exception.

220. La clase Exception se encuentra en el namespace System.

221. Las dos clases genéricas más importantes definidas en el Framework además de Exception son System.Object y System.ValueType.

222. La instrucción utilizada para poner en práctica el control e interceptar las excepciones es try-catch (con el bloque finally opcional).

223. El código protegido contra excepciones se coloca dentro del bloque try.

224. Para interceptar un error de división por cero se utiliza la excepción DivideByZeroException.

225. Para interceptar una DLL que tiene problemas al ser cargada se utiliza la excepción DllNotFoundException.

226. Para especificar una condición adicional dentro de una cláusula Catch, se utiliza la cláusula when (en C#) o When (en VB), que evalúa una condición booleana adicional para determinar si ese bloque catch debe ejecutarse.

227. El código de limpieza y liberación de recursos que debe ejecutarse siempre (ocurra o no una excepción) se coloca dentro del bloque finally.

228. La instrucción utilizada para provocar un error y adaptarlo al mecanismo de control de excepciones es throw.

229. El código para provocar una excepción del tipo ArgumentException sería: throw new ArgumentException("Argumento inválido");

230. Para construir un objeto del tipo Exception personalizado se define una clase que herede de Exception, se implementan constructores que llamen a los de la clase base, y opcionalmente se agregan propiedades adicionales para información del error. Ejemplo: public class ClienteNoExisteException : Exception { public ClienteNoExisteException(string msg) : base(msg) { } }

231. Un Catch personalizado para la excepción ClienteNoExisteException se armaría así: catch (ClienteNoExisteException ex) { /* tratamiento específico */ }

232. Las instancias de los objetos administrados por el GC se almacenan en el Managed Heap (montículo administrado).

233. Los dos métodos más notorios para trabajar correctamente con la recolección de elementos no utilizados son: Finalize() (el destructor, llamado por el GC) y Dispose() (de la interfaz IDisposable, llamado explícitamente por el programador).

234. El método utilizado para que el GC recolecte los elementos no utilizados es GC.Collect().

235. El método utilizado para suspender el subproceso actual hasta que el subproceso de finalizadores vacíe su cola es GC.WaitForPendingFinalizers().

236. Cuando se ejecuta el método GC.Collect(), se ejecutan los métodos Finalize() (destructores) de los objetos que poseen finalizador y que van a ser recolectados.

237. Una clase bien diseñada que no posee destructor debería exponer el método Dispose() implementando la interfaz IDisposable, para permitir la liberación explícita de recursos.

238. El método Dispose() se obtiene implementando la interfaz IDisposable, que obliga a definir el método public void Dispose().

239. En el método Dispose() se programa la liberación de recursos como: cerrar archivos y conexiones a bases de datos, liberar memoria no administrada, liberar handles del sistema operativo y en general limpiar cualquier recurso externo utilizado por el objeto.

240. Sí, se pueden combinar Dispose() y el destructor. El patrón recomendado es que Dispose() libere los recursos inmediatamente cuando es llamado explícitamente, y que el destructor llame a Dispose() como mecanismo de seguridad por si el programador olvidó llamarlo.

241. La resurrección de objetos ocurre cuando un objeto que iba a ser recolectado por el GC vuelve a ser referenciado durante la ejecución de su método Finalize(), lo que hace que el GC lo rescate de la recolección.

242. Una generación, en el contexto de la recolección de elementos no utilizados, es una clasificación que el GC asigna a los objetos en memoria para optimizar el proceso de recolección según su antigüedad.

243. La generación de un objeto puede adoptar los valores: 0 (objetos recién creados), 1 (objetos que sobrevivieron una recolección) y 2 (objetos que sobrevivieron más de una recolección, los más longevos).

244. Para obtener el número de veces que se ha producido la recolección de elementos no utilizados para una generación específica se utiliza GC.CollectionCount(int generation).

245. Para obtener el número de generación actual de un objeto se utiliza GC.GetGeneration(objeto).

246. Para recuperar el número de bytes que se considera que están asignados actualmente en el Managed Heap se utiliza GC.GetTotalMemory(bool forceFullCollection).

247. Para convertir un objeto en no válido para la recolección de elementos no utilizados desde el principio de la rutina actual hasta el momento en que se llamó al método, se utiliza GC.KeepAlive(objeto).

248. Para solicitar que el sistema no llame al finalizador del objeto especificado se utiliza GC.SuppressFinalize(objeto).

249. Para solicitar que el sistema vuelva a llamar al finalizador del objeto especificado, para el que previamente se llamó a SuppressFinalize, se utiliza GC.ReRegisterForFinalize(objeto).

250. Para obtener el número máximo de generaciones que el sistema admite actualmente se utiliza GC.MaxGeneration.

UNIDAD IV

251. Las interfaces se utilizan para definir un contrato de comportamiento que las clases deben implementar, permitiendo el polimorfismo sin necesidad de herencia de implementación. Permiten que clases no relacionadas compartan un tipo común.

252. Una interfaz se implementa en una clase declarando la clase con la sintaxis: class MiClase : IInterfaz { } y luego implementando todos los miembros definidos en la interfaz.

253. Sí, las interfaces pueden heredar de otras interfaces, lo que permite crear jerarquías de contratos de comportamiento.

254. Sí, se puede implementar un tipo de polimorfismo peculiar por medio de interfaces: distintas clases sin relación de herencia entre sí pueden compartir un tipo común (la interfaz) y ser tratadas polimórficamente a través de ella.

255. La interfaz IComparable se utiliza para permitir que un objeto sea comparado con otro del mismo tipo, habilitando el ordenamiento de colecciones. Define el método CompareTo().

256. La interfaz IComparer se utiliza para definir criterios personalizados de comparación entre dos objetos, permitiendo ordenamientos alternativos al natural. Define el método Compare().

257. La interfaz ICloneable se utiliza para indicar que un objeto puede ser clonado. Define el método Clone() que debe devolver una copia del objeto.

258. La interfaz IEnumerable se utiliza para indicar que un objeto puede ser recorrido como una colección. Define el método GetEnumerator() que devuelve un enumerador.

259. La interfaz IEnumerator se utiliza para controlar el recorrido interno de una colección, permitiendo avanzar elemento a elemento. Define los métodos MoveNext(), Reset() y la propiedad Current.

260. Un delegado es un tipo que representa una referencia a un método con una firma específica. Permite tratar los métodos como objetos que pueden ser pasados como parámetros o asignados a variables.

261. Los delegados pueden delegar a métodos de instancia y a métodos estáticos de cualquier clase, siempre que su firma sea compatible con la del delegado.

262. Se puede delegar el comportamiento definido por cualquier método cuya firma (tipo de retorno y parámetros) sea compatible con la declaración del delegado.

263. Un delegado se construye declarando un tipo delegado con la firma del método: delegate TipoRetorno NombreDelegado(parámetros); y luego creando una instancia del delegado apuntando a un método: NombreDelegado d = new NombreDelegado(objeto.Metodo);

264. Un procedimiento de devolución de llamadas (callback) se implementa declarando un método que acepte un delegado como parámetro y lo invoque cuando corresponda, permitiendo que el llamante defina qué método se ejecutará.

265. La multidifusión de delegados sirve para que un único delegado invoque varios métodos encadenados en secuencia. Se logra combinando delegados con el operador += y se utiliza para implementar eventos con múltiples suscriptores.

UNIDAD V

266. Los tipos genéricos son tipos que se definen con parámetros de tipo, permitiendo crear clases, métodos, interfaces y delegados que funcionan con cualquier tipo de dato sin perder la seguridad de tipos. Se usan para escribir código reutilizable e independiente del tipo concreto.

267. Los genéricos se pueden utilizar a partir de la versión 2.0 de C#.

268. Las ventajas de utilizar genéricos son: reutilización de código, seguridad de tipos en tiempo de compilación, mejor rendimiento al evitar boxing/unboxing y conversiones innecesarias, y código más legible y mantenible.

269. Los genéricos se pueden aplicar a: clases, estructuras (structs), interfaces, métodos y delegados.

270. Los usos más comunes de los genéricos son: colecciones genéricas (List<T>, Dictionary<K,V>), estructuras de datos reutilizables (Stack<T>, Queue<T>), algoritmos de ordenamiento y búsqueda, y métodos utilitarios independientes del tipo.

271. Al crear clases genéricas se deben considerar: la independencia del parámetro de tipo, si se necesitan restricciones (constraints) sobre el tipo, la varianza del tipo, y qué operaciones del tipo se necesitarán dentro de la clase.

272. Heredar de una clase genérica abierta significa que la clase derivada mantiene el parámetro de tipo sin especificarlo (ej: class MiLista<T> : Lista<T>). Heredar de una clase genérica cerrada implica que el tipo ya está definido en la herencia (ej: class MiListaInt : Lista<int>).

273. Es importante colocar restricciones en los parámetros de tipo para limitar los tipos válidos y garantizar que tengan ciertas características necesarias para el funcionamiento de la clase o método genérico (ej: que implementen una interfaz, que sean tipos de referencia, o que tengan constructor sin parámetros).

274. Los tipos de restricciones que se pueden colocar a un parámetro de tipo son: where T : class (tipo de referencia), where T : struct (tipo de valor), where T : new() (tiene constructor sin parámetros), where T : NombreClase (hereda de esa clase), where T : IInterfaz (implementa esa interfaz).

275. Ejemplo de método genérico con un parámetro de tipo: public T ObtenerPrimero<T>(List<T> lista) { return lista[0]; } — este método puede recibir una lista de cualquier tipo T y devolver su primer elemento.

276. LINQ (Language Integrated Query) es una tecnología de .NET que permite realizar consultas sobre

distintos orígenes de datos utilizando una sintaxis integrada en el lenguaje, similar a las consultas SQL.

277. Con LINQ se pueden consultar: colecciones en memoria (LINQ to Objects), bases de datos (LINQ to SQL / Entity Framework), documentos XML (LINQ to XML), DataSets (LINQ to DataSet) y otros orígenes de datos.

278. Las partes básicas de una consulta LINQ son: el origen de datos (from), la expresión de consulta (where, select, orderby, etc.) y la ejecución de la consulta (que puede ser diferida o inmediata).

279. En una estructura LINQ la consulta especifica qué datos se desean obtener, cómo deben filtrarse (where), cómo ordenarse (orderby) y cómo proyectarse (select), es decir, qué forma tendrán los resultados.

280. Tres cláusulas importantes en una expresión de consulta LINQ son: from (define el origen y la variable de rango), where (establece condiciones de filtrado) y select (proyecta los resultados).

281. La cláusula from especifica el origen de datos y la variable de iteración; where establece la condición que deben cumplir los elementos para ser incluidos en el resultado; select define qué información se devuelve y en qué forma.

282. Algunos métodos de extensión de IEnumerable son: Count() (cuenta elementos), Where() (filtra), Select() (proyecta), OrderBy() (ordena ascendente), OrderByDescending() (ordena descendente), First() (primer elemento), FirstOrDefault() (primer elemento o valor por defecto), ToList() (materializa en lista), Sum(), Max(), Min(), Average(), GroupBy(), Join().

283. Para ordenar en una expresión LINQ se utiliza la cláusula orderby (en sintaxis de consulta) o los métodos OrderBy() y OrderByDescending() (en sintaxis de métodos de extensión).

284. Para lograr una unión entre dos orígenes de datos en una expresión LINQ se utiliza la cláusula join (en sintaxis de consulta) o el método de extensión Join() (en sintaxis fluida).

285. Para generar nuevos tipos en LINQ se utilizan tipos anónimos mediante la cláusula select con la sintaxis: select new { Propiedad1 = x.Campo1, Propiedad2 = x.Campo2 }.

286. Una expresión lambda es una función anónima compacta que puede contener expresiones o sentencias, y que se puede usar en cualquier lugar donde se espere un delegado o una expresión de árbol.

287. Ejemplo de expresión lambda para realizar una consulta: var mayores = lista.Where(x => x.Edad > 18).Select(x => x.Nombre); — aquí x => x.Edad > 18 es una expresión lambda que filtra los elementos con edad mayor a 18.

288. Para crear una expresión lambda se define: una lista de parámetros entre paréntesis (o sin paréntesis si es uno solo), seguida del operador => (goes to / flecha), seguida de la expresión o bloque de sentencias a ejecutar.

289. Func es un tipo delegado genérico predefinido en .NET. Se declara como Func<T1, T2, ..., TResult> donde T1, T2... son los tipos de los parámetros de entrada y TResult es el tipo de retorno.

290. Un delegado Func con tres parámetros de entrada (int, double, bool) y retorno bool se declara así: Func<int, double, bool, bool> miDelegado = (a, b, c) => /* expresión que retorna bool */;

UNIDAD VI

291. Un esquema cliente-servidor es una arquitectura distribuida en la que existe al menos un proceso cliente que solicita servicios y al menos un proceso servidor que los provee. El servidor espera solicitudes, el cliente las inicia, y se comunican a través de una red.

292. Pasar información batch significa enviar o procesar la información en lotes, es decir, acumulando los datos y enviándolos en bloques sin interacción en tiempo real con el usuario.

293. Pasar información on line significa procesar o enviar la información en tiempo real, con interacción directa e inmediata entre el usuario y el sistema.

294. Un protocolo es un conjunto de reglas y convenciones que definen cómo se intercambia información entre dos o más partes que se comunican, especificando el formato, el orden de los mensajes y las acciones a tomar.

295. Las redes de área local (LAN) utilizan comúnmente el protocolo Ethernet como protocolo de acceso al medio.

296. Internet utiliza el protocolo TCP/IP (Transmission Control Protocol / Internet Protocol).

297. El protocolo IP (Internet Protocol) se encarga de identificar de forma única a cada dispositivo en la red mediante una dirección IP y de enrutar los paquetes de datos desde el origen hasta el destino.

298. Las ventajas de distribuir procesos son: mejor aprovechamiento de los recursos del sistema, mayor rendimiento mediante procesamiento paralelo, mayor disponibilidad y tolerancia a fallos, y escalabilidad de la solución.

299. Las ventajas de distribuir almacenamientos son: compartir información entre múltiples sistemas, mejorar la disponibilidad de los datos mediante redundancia, mayor escalabilidad del almacenamiento y posibilidad de acceso desde distintas ubicaciones.

300. Un socket es una combinación de dirección IP y número de puerto que identifica un extremo de una comunicación en red, permitiendo establecer conexiones y transferir datos entre aplicaciones.

301. Un socket sincrónico se caracteriza porque el proceso que lo utiliza queda bloqueado esperando que la operación (enviar o recibir) se complete antes de continuar su ejecución.

302. Un socket asincrónico se caracteriza porque el proceso que lo utiliza puede continuar ejecutándose sin esperar que la operación de red se complete; se notifica mediante callbacks o eventos cuando la operación finaliza.

303. Para construir un navegador o cliente web se puede utilizar el objeto WebClient o HttpWebRequest (en .NET Framework) o HttpClient (en .NET moderno).

304. Los puertos de la PC se utilizan para permitir la comunicación entre distintas aplicaciones o servicios en un mismo dispositivo o entre dispositivos, identificando qué aplicación debe recibir cada conjunto de datos.

305. Una PC puede poseer puertos físicos (serie, paralelo, USB, HDMI, Ethernet, audio) y puertos lógicos de red (numerados del 0 al 65535, donde los bien conocidos son del 0 al 1023).

306. El puerto paralelo transmite varios bits de información simultáneamente a través de múltiples líneas de datos en paralelo, lo que lo hacía rápido para impresoras y dispositivos de almacenamiento externos.

307. El puerto serie transmite la información bit por bit de forma secuencial a través de una sola línea de datos, siendo más lento pero más simple y confiable para comunicaciones a larga distancia.

308. El puerto USB (Universal Serial Bus) permite la conexión e intercambio de datos de manera universal

y estandarizada con múltiples dispositivos, con características de plug and play, alimentación eléctrica y alta velocidad de transferencia.

309. La domótica es la automatización integral de viviendas y edificios mediante el uso de tecnologías informáticas y de comunicación que controlan y gestionan sistemas como iluminación, seguridad, climatización, electrodomésticos y otros sistemas del hogar, mejorando la comodidad, eficiencia y seguridad.

GUÍA DE ABORDAJE BIBLIOGRÁFICO

UNIDAD I

Cardacci Dario y Booch, Grady. **Orientación a Objetos. Teoría y Práctica.** -- Buenos Aires, Argentina. Pearson Argentina, 2013. Capítulos 1 al 4.

Deitel Harvey M. Y Paul J. Deitel. Cómo programar en C#. Segunda edición. Pearson. México 2007. Capítulos 9 al 11

UNIDAD II

Cardacci Dario y Booch, Grady. **Orientación a Objetos. Teoría y Práctica.** -- Buenos Aires, Argentina. Pearson Argentina, 2013. Capítulos 5.1 al 5.6

Deitel Harvey M. Y Paul J. Deitel. Cómo programar en C#. Segunda edición. Pearson. México 2007. Capítulos 9 al 11

UNIDAD III

https://docs.microsoft.com/es-es/dotnet/csharp/language-reference/keywords/exception-handling-statements

Balena, Francesco. **Programación avanzada con Visual Basic 2005.** -- México, DF: McGraw-Hill Interamericana, c2008. Capítulo 1.

Deitel Harvey M. Y Paul J. Deitel. Cómo programar en C#. Segunda edición. Pearson. México 2007. Capítulos 12.

UNIDAD IV

https://docs.microsoft.com/es-es/dotnet/csharp/tour-of-csharp/interfaces

https://docs.microsoft.com/es-es/dotnet/csharp/tour-of-csharp/delegates

UNIDAD V

Deitel Harvey M. Y Paul J. Deitel. Cómo programar en C#. Segunda edición. Pearson. México 2007. Capítulos 25

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/concepts/linq/

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/generics/

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/concepts/linq/linq-and-generic-types

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/statements-expressions-operators/lambda-expressions

UNIDAD VI

MSDN. Microsoft Developer Network.
http://msdn.microsoft.com/en-us/library/system.net.sockets.socket.aspx

Cardacci Dario y Booch, Grady. **Orientación a Objetos. Teoría y Práctica.** -- Buenos Aires, Argentina. Pearson Argentina, 2013. Capítulos 5.7.

Deitel Harvey M. Y Paul J. Deitel. Cómo programar en C#. Segunda edición. Pearson. México 2007. Capítulos 23

Montefinal, Fabián H.; Cardacci, Darío G. **Conceptos básicos sobre electricidad: electrónica y puertos de la PC**. 2a.ed.-- Buenos Aires: Universidad Abierta Interamericana, c2006. 78 páginas